

Module Niveau 3

IA Générative Avancée : RAG, Agents et Fine-Tuning

Durée : 35h (5 jours)

Public : Développeurs, Data Scientists, Ingénieurs, Architectes logiciels, Chefs de projet techniques

Certification : "Expert en Ingénierie des Systèmes d'IA Générative"

Version : Août 2026

Académie de Prompt Engineering

De Novice à Expert en IA Générative | Public : Toutes personnes sachant lire et écrire utilisant un smartphone

Table des Matières

Module Niveau 3 : IA Générative Avancée : RAG, Agents et Fine-Tuning

Fiche Module

- [Public cible](#), [Prérequis](#), [Compétences visées](#), [Valeur marchande](#)

Partie Théorique (14h)

- [1. Architecture des Transformeurs en Profondeur](#)
 - Mécanismes d'attention et positionnement
 - Normalisation et résidual connections
- [2. Embeddings et Recherche Vectorielle](#)
 - Similitude cosinus et indexation
 - Vector Stores (FAISS, Pinecone, Weaviate)
- [3. RAG Avancé](#)
 - Ingestion, découpage (chunking), retrieval
 - Stratégies d'amélioration
- [4. Agents Autonomes](#)
 - Raisonnement (ReAct)
 - Appels d'outils et orchestration
 - Boucles et mémoire
- [5. Fine-Tuning \(LoRA, PEFT\)](#)
 - Jeux de données et évaluation
 - Risque de sur-apprentissage
- [6. Évaluation, Benchmarks et Sécurité](#)
 - Métriques et tests adverses
 - Coûts (tokens, latence)
 - Prompt injection et fuite de données
 - Garde-fous et gouvernance
- [7. Lexique Technique du Niveau 3](#)

Partie Pratique (16h)

- [1. Ateliers Guidés \(avec code\)](#)
 - Atelier 1 : Pipeline RAG minimal
 - Atelier 2 : Amélioration du retrieval
 - Atelier 3 : Agent avec appels d'outils
 - Atelier 4 : Fine-tuning LoRA
 - Atelier 5 : Évaluation automatisée et garde-fous
- [2. Études de Cas Réels](#)
 - Cas 1 : Assistant de support client sur base documentaire (RAG)
 - Cas 2 : Agent d'analyse de documents financiers
 - Cas 3 : Génération de rapports au format maison (fine-tuning)
- [3. Templates de Prompts Réutilisables](#)
 - Template Prompt système RAG
 - Template Agent avec planification
 - Template Évaluation

Partie Évaluation (5h)

- [1. QCM de Contrôle \(40 questions\)](#)
- [2. Projet Fil Rouge : Assistant métier complet](#)
- [3. Certification et Valorisation](#)

Valorisation

- [CV, GitHub, Positionnement client](#)

Ressources Supplémentaires

- [Lectures, Outils, Fiches Mémo, Prochaines Étapes](#)

Fiche Module - Niveau 3

Durée : 35h (5 jours) | **Public cible :** Développeurs, Data Scientists, Ingénieurs, Architectes logiciels, Chefs de projet techniques.

Prérequis

- Niveau 2 validé
- Bases de programmation (Python recommandé)
- Notions d'API et de bases de données

Titre du certificat

« **Certificat d'Expert en Ingénierie des Systèmes d'IA Générative** »

Compétences métiers visées

- Concevoir un système RAG (Retrieval-Augmented Generation) complet
- Construire des agents autonomes (planification, outils, boucles)
- Adapter un modèle par fine-tuning (LoRA, PEFT)
- Évaluer, sécuriser et déployer un système d'IA générative
- Estimer les coûts et arbitrer entre les architectures

Valeur marchande

Profil rare et très bien rémunéré ; missions de conseil et d'intégration ; capacité à porter des projets d'entreprise de bout en bout.

Positionnement : Sur le CV, mentionnez « Conception de systèmes RAG et d'agents autonomes ; fine-tuning LoRA ; évaluation et sécurisation de pipelines d'IA générative ». Sur GitHub, publiez le projet avec README, schéma d'architecture et captures. Après des clients, positionnez-vous sur des missions types « mettre en place un assistant RAG sur la base documentaire » ou « automatiser un processus avec des agents » ; tarification au projet ou en régie.



Partie Théorique

Durée : 14h | **Objectif :** Maîtriser les concepts avancés pour concevoir et déployer des systèmes d'IA générative complets.

1. Architecture des Transformeurs en Profondeur

Durée : 2h

1.1 Mécanismes d'Attention

Compréhension détaillée du mécanisme d'attention qui permet aux transformeurs de pondérer l'importance des différents tokens dans une séquence. Explication des concepts de **Query, Key, Value** et du calcul des scores d'attention.

Formule d'attention : $Attention(Q, K, V) = softmax(QK^T/\sqrt{d_k})V$

Où $Q = Query$, $K = Key$, $V = Value$, $d_k = dimension\ des\ clés$

1.2 Positionnement et Encodage Positionnel

Les transformeurs n'ont pas de notion innée de l'ordre des tokens. L'encodage positionnel ajoute des informations sur la position de chaque token dans la séquence, permettant au modèle de comprendre l'ordre.

1.3 Normalisation et Connexions Résiduelles

La normalisation de couche (Layer Normalization) et les connexions résiduelles sont essentielles pour stabiliser l'entraînement des réseaux profonds. Elles permettent de propager le gradient plus efficacement.

Pourquoi c'est important : Ces mécanismes permettent aux transformeurs de traiter des séquences très longues et d'apprendre des dépendances à longue portée.

2. Embeddings et Recherche Vectorielle

Durée : 2h

2.1 Embeddings : Représentation Vectorielle

Les embeddings sont des représentations vectorielles denses de données (texte, images, etc.). Ils capturent la sémantique et permettent des comparaisons efficaces.

2.2 Similitude Cosinus

Métrique utilisée pour mesurer la similarité entre deux vecteurs. Plus le score est proche de 1, plus les vecteurs sont similaires.

Formule : $similarity = (A \cdot B) / (||A|| \cdot ||B||)$

2.3 Vector Stores

Bases de données spécialisées pour le stockage et la recherche de vecteurs à grande échelle.

Outil	Description	Cas d'usage
FAISS	Bibliothèque Facebook pour recherche de similarité	Recherche rapide en mémoire
Pinecone	Service managé pour vecteurs	Applications en production
Weaviate	Base de données vectorielle open source	Recherche hybride (vecteurs + filtres)
Chroma	Base de données vectorielle open source	Prototypage et petits projets

3. RAG (Retrieval-Augmented Generation) Avancé

Durée : 3h

3.1 Principe du RAG

Le RAG combine la génération de texte avec la récupération d'informations externes. Cela permet de réduire les hallucinations et d'incorporer des connaissances spécifiques au domaine.

Architecture de base : *Ingestion* → *Découpage (Chunking)* → *Indexation* → *Retrieval* → *Génération*

3.2 Ingestion et Découpage (Chunking)

Le processus de préparation des documents : nettoyage, découpage en chunks de taille optimale, ajout de métadonnées.

Stratégies de Chunking

- **Chunking par taille fixe :** Découper en segments de N tokens
- **Chunking par paragraphe :** Utiliser les sauts de ligne comme délimiteurs
- **Chunking sémantique :** Utiliser des algorithmes pour identifier les limites naturelles
- **Chunking avec chevauchement :** Permettre un recouvrement entre chunks pour éviter de couper les informations

3.3 Retrieval

Récupération des chunks les plus pertinents pour une requête donnée. Utilisation de la recherche vectorielle avec des embeddings.

3.4 Génération

Le modèle génère une réponse en utilisant à la fois la requête et les chunks récupérés comme contexte.

Pourquoi le RAG réduit les hallucinations : En fournissant des informations factuelles externes, le modèle a moins tendance à inventer des informations.

3.5 Stratégies d'Amélioration

- **Re-ranking** : Réorganiser les résultats de la recherche pour améliorer la pertinence
- **Query expansion** : Élargir la requête pour capturer plus d'informations
- **Multi-hop retrieval** : Effectuer plusieurs étapes de recherche pour des requêtes complexes
- **Fusion de résultats** : Combiner les résultats de plusieurs requêtes

4. Agents Autonomes

Durée : 3h

4.1 Raisonnement avec ReAct

ReAct (Reasoning and Acting) combine le raisonnement en langage naturel avec des actions pour permettre aux agents de résoudre des problèmes complexes.

Format ReAct : *Pensée → Action → Observation → Pensée → Action...*

4.2 Appels d'Outils

Les agents peuvent utiliser des outils externes (APIs, fonctions) pour effectuer des tâches spécifiques. Cela étend leurs capacités au-delà de la simple génération de texte.

Exemples d'outils

- **Recherche web :** Accéder à des informations en temps réel
- **Calcul :** Effectuer des calculs mathématiques complexes
- **APIs spécialisées :** Accéder à des services externes (météo, bases de données, etc.)
- **Manipulation de fichiers :** Lire, écrire, modifier des fichiers

4.3 Orchestration

Coordination de plusieurs agents ou outils pour accomplir une tâche complexe. Cela peut impliquer des boucles, des conditions, et une planification.

4.4 Boucles et Mémoire

Les agents peuvent maintenir un état (mémoire) entre les interactions et effectuer des actions de manière itérative jusqu'à atteindre un objectif.

Comment un agent enchaîne raisonnement et actions : L'agent utilise le raisonnement pour décider quelle action effectuer, exécute l'action,

observe le résultat, puis utilise cette observation pour le raisonnement suivant.

5. Fine-Tuning (LoRA, PEFT)

Durée : 2h

5.1 Qu'est-ce que le Fine-Tuning ?

Le fine-tuning est le processus d'entraînement supplémentaire d'un modèle pré-entraîné sur un jeu de données spécifique pour l'adapter à une tâche particulière.

Différence avec le RAG : *Le fine-tuning modifie les poids du modèle pour changer son comportement, tandis que le RAG fournit des informations externes sans modifier le modèle.*

5.2 LoRA (Low-Rank Adaptation)

Technique de fine-tuning efficace qui ne met à jour qu'une petite partie des poids du modèle, réduisant ainsi les coûts de calcul et de mémoire.

5.3 PEFT (Parameter-Efficient Fine-Tuning)

Approches générales pour le fine-tuning efficace, incluant LoRA, les adapters, et d'autres méthodes.

5.4 Jeux de Données

Préparation des données pour le fine-tuning : formatage, nettoyage, augmentation, et division en ensembles d'entraînement, de validation et de test.

5.5 Évaluation

Mesurer les performances du modèle fine-tuné sur des métriques pertinentes pour la tâche.

Risque de sur-apprentissage : **Le modèle peut trop s'adapter aux données d'entraînement et perdre sa capacité à généraliser. Utilisez des techniques de régularisation et une validation croisée.**

5.6 Ce que le Fine-Tuning change

- **Style :** Le modèle peut adopter un style d'écriture spécifique
- **Format :** Le modèle peut générer des sorties dans un format particulier

- **Connaissances** : Le modèle peut acquérir de nouvelles connaissances dans un domaine spécifique
- **Comportement** : Le modèle peut suivre des instructions ou des contraintes spécifiques

À noter : *Le fine-tuning ne change pas les connaissances générales du modèle, mais plutôt son comportement et son style pour des tâches spécifiques.*

6. Évaluation, Benchmarks et Sécurité

Durée : 2h

6.1 Benchmarks

Évaluation des modèles sur des jeux de tests standardisés pour comparer les performances.

6.2 Métriques

Métriques courantes pour évaluer les systèmes d'IA générative :

- **Exactitude (Accuracy)** : Pourcentage de réponses correctes
- **Précision (Precision)** : Pourcentage de réponses positives correctes
- **Rappel (Recall)** : Pourcentage de toutes les réponses positives trouvées
- **F1 Score** : Moyenne harmonique de la précision et du rappel
- **BLEU** : Évaluation de la qualité des traductions
- **ROUGE** : Évaluation de la qualité des résumés

6.3 Tests Adverses

Tests conçus pour évaluer la robustesse du système face à des entrées malveillantes ou ambiguës.

6.4 Coûts

Estimation des coûts associés :

- **Tokens** : Coût par token pour l'entraînement et l'inférence
- **Latence** : Temps de réponse acceptable pour l'application
- **Infrastructure** : Coût des ressources matérielles et logicielles

6.5 Sécurité

Mesures de sécurité pour protéger le système et les utilisateurs :

Prompt Injection : Attaque où un utilisateur malveillant tente d'injecter des instructions malveillantes dans le prompt. Utilisez des techniques de filtrage et de validation pour vous protéger.

Fuite de données : Risque que le système divulgue des informations sensibles. Utilisez des garde-fous et limitez l'accès aux données sensibles.

6.6 Garde-fous et Gouvernance

Mise en place de mécanismes de contrôle et de gouvernance pour assurer un déploiement éthique et sécurisé :

- **Filtrage de contenu** : Bloquer les contenus inappropriés ou dangereux
- **Limites de taux** : Limiter le nombre de requêtes par utilisateur
- **Journalisation** : Enregistrer les interactions pour l'audit et le débogage
- **Contrôle d'accès** : Limiter l'accès aux fonctionnalités sensibles

7. Lexique Technique du Niveau 3

Durée : 1h

Terme	Définition
Transformeur	Architecture de réseau de neurones basée sur le mécanisme d'attention
Attention	Mécanisme permettant de pondérer l'importance des différents éléments d'une séquence
Embedding	Représentation vectorielle dense de données
Vector Store	Base de données spécialisée pour le stockage et la recherche de vecteurs
Chunking	Processus de découpage de documents en segments de taille gérable
RAG	Retrieval-Augmented Generation : génération augmentée par récupération
Agent	Système capable de raisonner et d'agir de manière autonome
ReAct	Reasoning and Acting : approche combinant raisonnement et actions
Outil (Tool)	Fonction externe que l'agent peut appeler pour effectuer une tâche
Fine-tuning	Entraînement supplémentaire d'un modèle pré-entraîné sur une tâche spécifique
LoRA	Low-Rank Adaptation : technique de fine-tuning efficace
PEFT	Parameter-Efficient Fine-Tuning : approches pour un fine-tuning efficace
Benchmark	Jeu de tests standardisé pour évaluer les performances
Token	Unité de base de traitement pour les modèles de langage
Latence	Temps de réponse du système

Terme	Définition
Garde-fou	Mécanisme de protection contre les comportements indésirables
Prompt Injection	Attaque par injection d'instructions malveillantes
MLOps	Pratiques pour le déploiement et la maintenance des modèles de ML en production

Mécanismes sous-jacents

- **Pourquoi le RAG réduit les hallucinations** : En fournissant des informations factuelles externes, le modèle a moins tendance à inventer des informations.
- **Comment un agent enchaîne raisonnement et actions** : L'agent utilise le raisonnement pour décider quelle action effectuer, exécute l'action, observe le résultat, puis utilise cette observation pour le raisonnement suivant.
- **Ce que le fine-tuning change réellement** : Principalement le style et le format des sorties, ainsi que le comportement pour des tâches spécifiques, plutôt que les connaissances générales.

✂ Partie Pratique

Durée : 16h | **Objectif :** Mettre en pratique les concepts théoriques à travers des ateliers guidés et des études de cas réels.

1. Ateliers Guidés (avec code)

Durée : 10h

Atelier 1 : Construire un Pipeline RAG Minimal

Durée : 2h | **Objectif :** Créer un pipeline RAG fonctionnel de base.

Étapes :

- Préparation de l'environnement :** Installer les bibliothèques nécessaires (LangChain, FAISS, sentence-transformers)
- Ingestion des documents :** Charger et nettoyer les documents source
- Découpage (Chunking) :** Découper les documents en chunks de taille appropriée
- Indexation :** Créer un index de recherche vectorielle avec FAISS
- Retrieval :** Implémenter la fonction de récupération des chunks pertinents
- Génération :** Intégrer un LLM pour générer des réponses basées sur les chunks récupérés
- Test :** Évaluer le pipeline avec des requêtes de test

Code exemple (Python) :

```
from langchain.document_loaders import DirectoryLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import FAISS
from langchain.llms import HuggingFaceHub
from langchain.chains import RetrievalQA

# 1. Charger les documents
loader = DirectoryLoader('./documents/', glob="**/*.pdf")
documents = loader.load()
```

```
# 2. Découper en chunks
text_splitter = RecursiveCharacterTextSplitter(chunk_size=100)
texts = text_splitter.split_documents(documents)

# 3. Créer les embeddings et l'index
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers")
db = FAISS.from_documents(texts, embeddings)

# 4. Créer le retriever
retriever = db.as_retriever(search_kwargs={"k": 4})

# 5. Initialiser le LLM
llm = HuggingFaceHub(repo_id="google/flan-t5-xl", model_kwargs={})

# 6. Créer la chaîne RAG
qa = RetrievalQA.from_chain_type(llm=llm, chain_type="stuff", retriever=retriever)

# 7. Poser une question
query = "Quelle est la politique de remboursement ?"
result = qa.run(query)
print(result)
```

Atelier 2 : Améliorer le Retrieval

Durée : 2h | **Objectif :** Optimiser la qualité de la récupération.

Stratégies à implémenter :

1. **Chunking avancé :** Expérimenter avec différentes tailles de chunks et stratégies de chevauchement
2. **Métadonnées :** Ajouter des métadonnées utiles (type de document, date, auteur) et les utiliser pour filtrer
3. **Re-ranking :** Implémenter un système de re-ranking pour améliorer la pertinence
4. **Query expansion :** Élargir automatiquement les requêtes pour capturer plus d'informations
5. **Évaluation :** Créer un jeu de tests pour évaluer la qualité du retrieval

Atelier 3 : Créer un Agent avec Appels d'Outils

Durée : 2h | **Objectif :** Construire un agent capable d'utiliser des outils externes.

Étapes :

1. **Définir les outils :** Créer des fonctions pour les outils que l'agent pourra utiliser (recherche web, calcul, accès à une API)
2. **Créer l'agent :** Utiliser LangChain ou une autre bibliothèque pour créer l'agent
3. **Configurer le raisonnement :** Définir comment l'agent doit raisonner et décider quels outils utiliser
4. **Intégrer les outils :** Connecter les outils à l'agent
5. **Tester :** Évaluer l'agent avec des tâches complexes

Exemple d'outils :

```
# Exemple avec LangChain
from langchain.agents import load_tools, initialize_agent
from langchain.llms import OpenAI

# Charger les outils (ex: calculatrice, recherche web)
tools = load_tools(["llm-math", "serpapi"], llm=llm)

# Initialiser l'agent
agent = initialize_agent(tools, llm, agent="zero-shot-react-d

# Exécuter une tâche
result = agent.run("Quelle est la météo à Paris aujourd'hui ?")
print(result)
```

Atelier 4 : Fine-Tuning LoRA d'un Petit Modèle

Durée : 2h | **Objectif :** Adapter un modèle à une tâche spécifique avec LoRA.

Étapes :

1. **Préparer les données :** Créer ou obtenir un jeu de données pour la tâche cible
2. **Prétraitement :** Nettoyer et formater les données pour le fine-tuning
3. **Configurer LoRA :** Définir les paramètres de LoRA (rank, alpha, etc.)
4. **Entraîner :** Effectuer le fine-tuning avec les données préparées
5. **Évaluer :** Tester le modèle fine-tuné sur un jeu de validation
6. **Comparer :** Comparer les performances avec le modèle de base

Exemple avec PEFT :

```
from peft import LoraConfig, get_peft_model
from transformers import AutoModelForCausalLM, AutoTokenizer

# Charger le modèle et le tokenizer
model_name = "t5-small"
model = AutoModelForCausalLM.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)

# Configurer LoRA
lora_config = LoraConfig(
    r=8, # rank
    lora_alpha=16,
    target_modules=["q", "v"],
    lora_dropout=0.1,
    bias="none",
    task_type="CAUSAL_LM"
)

# Appliquer LoRA au modèle
model = get_peft_model(model, lora_config)
```

```
# Préparer les données et entraîner...  
# (code d'entraînement standard)
```

Atelier 5 : Mettre en Place une Évaluation Automatisée et des Garde-fous

Durée : 2h | **Objectif :** Assurer la qualité et la sécurité du système.

Étapes :

1. **Créer des tests :** Développer des tests automatisés pour évaluer les performances
2. **Métriques :** Définir et implémenter des métriques d'évaluation pertinentes
3. **Tests adverses :** Créer des tests pour évaluer la robustesse face à des entrées malveillantes
4. **Garde-fous :** Implémenter des mécanismes de protection (filtrage de contenu, limites de taux)
5. **Journalisation :** Mettre en place un système de journalisation pour l'audit
6. **Intégration continue :** Automatiser l'exécution des tests dans un pipeline CI/CD

2. Études de Cas Réels

Durée : 4h

Cas 1 : Assistant de Support Client sur une Base Documentaire Interne (RAG)

Contexte : Une entreprise souhaite mettre en place un assistant pour son équipe de support client, capable de répondre aux questions des clients en se basant sur la documentation interne.

Objectifs :

- Créer un pipeline RAG pour indexer la documentation client
- Implémenter un système de récupération précis
- Intégrer un LLM pour générer des réponses claires et utiles
- Évaluer la qualité des réponses
- Déployer le système pour l'équipe de support

Livrables :

- Pipeline RAG fonctionnel
- Interface pour le support client
- Documentation technique
- Rapport d'évaluation

Cas 2 : Agent d'Analyse de Documents Financiers avec Appels d'Outils

Contexte : Un cabinet de conseil financier souhaite automatiser l'analyse de documents financiers (bilans, comptes de résultat) pour identifier des tendances et générer des rapports.

Objectifs :

- Créer un agent capable de lire et comprendre les documents financiers
- Implémenter des outils pour extraire et analyser les données
- Développer un système de génération de rapports automatisés
- Assurer la précision et la fiabilité des analyses

Outils nécessaires :

- Outil d'extraction de texte à partir de PDF
- Outil de calcul financier
- Outil de génération de graphiques
- Outil de validation des données

Livrables :

- Agent fonctionnel avec outils intégrés
- Système de génération de rapports
- Documentation utilisateur
- Jeu de tests de validation

Cas 3 : Génération de Rapports au Format Maison (Fine-Tuning)

Contexte : Une entreprise a besoin de générer des rapports internes dans un format très spécifique, avec un style et une structure particuliers.

Objectifs :

- Fine-tuner un modèle pour générer des rapports dans le format souhaité
- Préparer un jeu de données d'exemples de rapports
- Évaluer la qualité des rapports générés
- Déployer le modèle fine-tuné

Étapes :

1. Collecter des exemples de rapports existants
2. Nettoyer et formater les données
3. Configurer et effectuer le fine-tuning
4. Évaluer les performances du modèle
5. Intégrer le modèle dans le système de génération de rapports

Livrables :

- Modèle fine-tuné
- Système de génération de rapports
- Documentation technique
- Rapport d'évaluation

3. Templates de Prompts Réutilisables

Durée : 2h

Template « Prompt Système RAG »

Utilisation : Pour configurer un assistant RAG avec des instructions claires.

Tu es un assistant expert sur [domaine]. Ta mission est de répondre

Règles :

- Si l'information demandée n'est pas présente dans le contexte,
- Ne devine pas et n'invente pas d'informations.
- Sois précis et concis.
- Cite la source lorsque c'est pertinent.

Contexte :

[Insérer le contexte récupéré ici]

Question : [Question de l'utilisateur]

Réponse :

Template « Agent avec Planification »

Utilisation : Pour guider un agent dans la décomposition et l'exécution d'une tâche complexe.

Tu es un agent intelligent capable de raisonner et d'agir. Pour

1. **Analyse** : Comprends la demande et identifie les sous-tâches nécessaires.
2. **Planification** : Décompose la demande en étapes claires et logiques.
3. **Outils** : Identifie quels outils sont nécessaires pour chaque étape.
4. **Exécution** : Exécute les étapes dans l'ordre, en utilisant les outils appropriés.
5. **Vérification** : Vérifie que chaque étape a été exécutée correctement.
6. **Résultat final** : Fournis le résultat final à l'utilisateur.

Demande : [Demande de l'utilisateur]

Outils disponibles : [Liste des outils avec descriptions]

Plan :

Template « Évaluation »

Utilisation : Pour évaluer de manière systématique les réponses générées.

Évalue la réponse générée par rapport au corrigé attendu sur les

Critères d'évaluation :

1. **Exactitude** : La réponse est-elle factuellement correcte ? (0-10)
2. **Exhaustivité** : La réponse couvre-t-elle tous les aspects de la question ? (0-10)
3. **Format** : La réponse suit-elle le format demandé ? (0-10)
4. **Clarté** : La réponse est-elle claire et facile à comprendre ? (0-10)
5. **Pertinence** : La réponse est-elle pertinente par rapport à la question ? (0-10)

Corrigé attendu : [Insérer le corrigé ici]

Réponse générée : [Insérer la réponse ici]

Évaluation :

- Exactitude : [Note]/10 - Justification : [Commentaire]
- Exhaustivité : [Note]/10 - Justification : [Commentaire]
- Format : [Note]/10 - Justification : [Commentaire]
- Clarté : [Note]/10 - Justification : [Commentaire]
- Pertinence : [Note]/10 - Justification : [Commentaire]

Note totale : [Somme]/50

Commentaires généraux : [Commentaires]

Partie Évaluation

Durée : 5h | **Objectif :** Valider les compétences acquises à travers un QCM et un projet pratique.

1. QCM de Contrôle

Durée : 60 minutes | **Nombre de questions :** 40

Thèmes couverts :

- Architecture des transformeurs (8 questions)
- Embeddings et recherche vectorielle (6 questions)
- RAG (8 questions)
- Agents autonomes (6 questions)
- Fine-tuning (6 questions)
- Sécurité et évaluation (6 questions)

Format : Questions à choix multiples, questions vrai/faux, questions courtes.

Seuil de réussite : 70% (28/40)

Exemples de Questions

Architecture des Transformeurs

1. **Quelle est la formule de base du mécanisme d'attention ?**

- A) $\text{Attention}(Q, K, V) = QK^T V$
- B) $\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k})V$
- C) $\text{Attention}(Q, K, V) = (Q + K)V$
- D) $\text{Attention}(Q, K, V) = QV^T K$

Réponse : B) $\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k})V$

2. **À quoi sert l'encodage positionnel dans les transformeurs ?**

- A) À accélérer le calcul
- B) À réduire la taille du modèle
- C) À fournir des informations sur l'ordre des tokens
- D) À filtrer les informations non pertinentes

Réponse : C) À fournir des informations sur l'ordre des tokens

RAG

1. Quels sont les 5 étapes principales d'un pipeline RAG ?

Réponse : Ingestion, Découpage (Chunking), Indexation, Retrieval, Génération

2. Pourquoi le RAG réduit-il les hallucinations ?

Réponse : Parce qu'il fournit des informations factuelles externes au modèle, réduisant ainsi la nécessité d'inventer des informations.

Agents

1. Qu'est-ce que ReAct ?

- A) Un modèle de langage
- B) Une approche combinant raisonnement et actions
- C) Une bibliothèque pour le fine-tuning
- D) Un outil de visualisation

Réponse : B) Une approche combinant raisonnement et actions

Fine-Tuning

1. Quelle est la différence principale entre le fine-tuning et le RAG ?

Réponse : Le fine-tuning modifie les poids du modèle, tandis que le RAG fournit des informations externes sans modifier le modèle.

2. Qu'est-ce que LoRA ?

- A) Un modèle de langage
- B) Une technique de fine-tuning efficace
- C) Une bibliothèque pour le RAG
- D) Un outil de visualisation

Réponse : B) Une technique de fine-tuning efficace

Sécurité

1. Qu'est-ce que le prompt injection ?

Réponse : Une attaque où un utilisateur malveillant tente d'injecter des instructions malveillantes dans le prompt pour manipuler le comportement du système.

2. **Citez deux mécanismes de protection contre les comportements indésirables.**

Réponse : Filtrage de contenu, limites de taux, journalisation, contrôle d'accès (au choix)

2. Projet Fil Rouge : Assistant Métier Complet

Durée : 4h | **Poids :** 70 points

Description du Projet

Concevoir et déployer un système d'IA générative (RAG et/ou agent) répondant à un besoin réel, avec évaluation, garde-fous et documentation complète.

Étapes du Projet

- Définition du besoin (1h) :** Identifier un problème concret à résoudre dans votre domaine professionnel ou personnel
- Conception de l'architecture (1h) :** Définir l'architecture du système (RAG, agent, ou combinaison)
- Développement (1.5h) :** Implémenter le système avec les technologies appropriées
- Évaluation et garde-fous (30 min) :** Mettre en place des tests et des mécanismes de sécurité
- Documentation (30 min) :** Rédiger une documentation technique complète
- Soutenance (20 min) :** Présenter le projet à l'oral

Livrables

- Code source :** Code complet et fonctionnel du système
- Documentation technique :** Description de l'architecture, des technologies utilisées, et des instructions d'installation
- Jeu de tests :** Tests automatisés pour évaluer les performances
- Rapport d'évaluation :** Résultats des tests et analyse des performances
- Présentation :** Support de présentation pour la soutenance orale

Grille d'Évaluation (70 points)

Critère	Points	Description
Pertinence de l'architecture	15	L'architecture est-elle adaptée au problème ? Utilisation appropriée de RAG, agents, etc.
Qualité du retrieval ou des agents	20	Les composants de retrieval (RAG) ou d'agents fonctionnent-ils correctement et produisent-ils des résultats de qualité ?

Critère	Points	Description
Évaluation et garde-fous	15	Les mécanismes d'évaluation et de sécurité sont-ils adéquats et efficaces ?
Code et documentation	10	Le code est-il propre, bien structuré et documenté ? La documentation est-elle complète ?
Soutenance	10	La présentation orale est-elle claire, bien structurée et convaincante ?

Seuil de Réussite

Global : 70/100 (QCM + Projet)

QCM : Minimum 70% (28/40)

Projet : Minimum 50/70

Conseils pour réussir :

- **Choisissez un projet réaliste** : Ne vous surchargez pas avec un projet trop complexe pour le temps imparti.
- **Commencez par le QCM** : Assurez-vous de bien réussir le QCM pour valider vos connaissances théoriques.
- **Documentez au fur et à mesure** : Ne laissez pas la documentation pour la fin.
- **Testez tôt et souvent** : Testez votre système régulièrement pour détecter les problèmes rapidement.
- **Préparez la soutenance** : Entraînez-vous à présenter votre projet de manière claire et concise.

3. Certification et Valorisation

Certificat

À l'issue réussie de l'évaluation, vous recevrez le « **Certificat d'Expert en Ingénierie des Systèmes d'IA Générative** ».

Compétences Certifiées

- Conception de systèmes RAG complets
- Construction d'agents autonomes
- Adaptation de modèles par fine-tuning (LoRA, PEFT)
- Évaluation et sécurisation de pipelines d'IA générative
- Estimation des coûts et arbitrage entre architectures

Reconnaissance

Ce certificat est reconnu par les employeurs et les clients comme une preuve de vos compétences avancées en IA générative.

Validité : *Le certificat est valable à vie, mais nous recommandons de suivre des formations de mise à jour régulièrement pour rester à jour avec les dernières avancées technologiques.*

Valorisation sur le Marché du Travail

Sur le CV

Exemple de formulation :

Expert en Ingénierie des Systèmes d'IA Générative
Certification : Académie de Prompt Engineering - Niveau 3

Compétences clés :

- Conception de systèmes RAG et d'agents autonomes
- Fine-tuning LoRA et PEFT
- Évaluation et sécurisation de pipelines d'IA générative
- Estimation des coûts et arbitrage architectural

Projet fil rouge : [Nom du projet] - [Brève description]
Lien vers le dépôt GitHub : [URL]

Sur GitHub

Publiez votre projet fil rouge avec :

- **README.md** : Description complète du projet, de l'architecture et des fonctionnalités
- **Schéma d'architecture** : Diagramme montrant les composants du système et leurs interactions
- **Captures d'écran** : Illustrations du fonctionnement du système
- **Documentation technique** : Instructions d'installation et d'utilisation
- **Code propre** : Code bien structuré, commenté et documenté

Pourquoi c'est important : Pour les recruteurs techniques, un dépôt GitHub bien documenté est la preuve la plus forte de vos compétences. C'est souvent le premier critère d'évaluation.

Auprès des Clients

Positionnement : Conseil et intégration en IA générative

Missions types :

- Mettre en place un assistant RAG sur une base documentaire interne
- Automatiser un processus métier avec des agents autonomes
- Adapter un modèle à un domaine spécifique par fine-tuning
- Évaluer et sécuriser un pipeline d'IA générative existant
- Former une équipe au développement de systèmes d'IA générative

Tarification :

- **Au projet :** Forfait pour la conception et le déploiement complet d'un système
- **En régie :** Tarif horaire ou journalier pour des missions de conseil ou de développement
- **Formation :** Tarif par personne pour des ateliers ou formations sur mesure

Exemple de tarifs (indicatifs) :

- **Projet RAG simple :** 5 000 - 15 000 €
- **Projet avec agents :** 10 000 - 30 000 €
- **Fine-tuning personnalisé :** 8 000 - 20 000 €
- **Audit et sécurisation :** 3 000 - 10 000 €
- **Formation (2 jours) :** 2 000 - 5 000 €

Note : Les tarifs varient selon la complexité, la taille de l'entreprise cliente, et la localisation géographique.



Ressources Supplémentaires

Lectures Recommandées

Livres

- **"Attention Is All You Need"** - Vaswani et al. (2017) - L'article fondateur sur les transformeurs
- **"Natural Language Processing with Transformers"** - Lewis Tunstall, Leandro von Werra, Thomas Wolf
- **"Deep Learning"** - Ian Goodfellow, Yoshua Bengio, Aaron Courville
- **"Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow"** - Aurélien Géron

Articles et Papers

- - Vaswani et al.
- - Lewis et al.
- - Yao et al.
- - Hu et al.

Outils et Bibliothèques

RAG

- - Framework pour le développement d'applications avec des LLM
- - Bibliothèque pour la recherche de similarité vectorielle
- - Service managé pour les vector stores
- - Base de données vectorielle open source

Agents

- - Framework pour la création d'agents
- - Framework Microsoft pour les agents conversants
- - Framework pour l'intégration de LLM avec des outils externes

Fine-Tuning

- - Bibliothèque pour le fine-tuning efficace
- - Bibliothèque pour l'entraînement et le fine-tuning de modèles
- - Implémentation de référence de LoRA

Fiches Mémo

Consultez les fiches mémo suivantes pour un rappel rapide des concepts clés :

-
-
-
-
-
-

Prochaines Étapes

Après avoir maîtrisé le Niveau 3, vous pouvez envisager les étapes suivantes :

- **Niveau 4 (Expert) :** Approfondir les sujets de recherche actuels (multi-modal, raisonnement avancé, alignement)
- **Contribuer à l'open source :** Participer au développement de bibliothèques comme LangChain, Hugging Face, etc.
- **Recherche :** Publier des articles ou travailler sur des projets de recherche innovants
- **Entrepreneuriat :** Créer votre propre startup dans le domaine de l'IA générative
- **Mentorat :** Devenir formateur ou mentor pour aider d'autres à monter en compétences

Communauté : Rejoignez notre groupe Discord/Slack pour échanger avec d'autres experts, partager des astuces, et rester informé des dernières actualités.

Résumé du Module

Niveau 3

Vous avez maintenant les compétences pour concevoir, développer et déployer des systèmes d'IA générative avancés.

Compétences Acquises

- ✓ Architecture des transformeurs
- ✓ Embeddings et recherche vectorielle
- ✓ RAG avancé
- ✓ Agents autonomes
- ✓ Fine-tuning (LoRA, PEFT)
- ✓ Évaluation et sécurité

Certification

- ✓ Certificat d'Expert
- ✓ QCM validé
- ✓ Projet fil rouge terminé
- ✓ Compétences prêtes pour le marché

Félicitations !

Vous êtes maintenant un·e Expert·e en Ingénierie des Systèmes d'IA Générative. Le monde a besoin de vos compétences !

